

A watchdog model for physics-based anomaly detection in digital substations

Hussam Tarazi*, Sara Sutton, John Olinjyk, Benjamin Bond, Julian Rrushi

School of Engineering and Computer Science Oakland University, Rochester Hills, Michigan 48309-4479, United States of America

ARTICLE INFO

Keywords:

Cyber physical system
Human machine interface
Industrial control system
Programmable logic controller
Watchdog
Sampled measure values
SCL programming language

ABSTRACT

The security of cyber-physical systems (CPS) presents new challenges stemming from computations that work primarily with live physics data. Although there is a body of previous research on detection of malware on CPS, more effective designs are needed to address limitations such as mimicry attacks and other forms of evasive techniques. Relay algorithms in particular, such as differential and harmonic protection algorithms, are essential to protecting physical equipment such as power transformers from faults. Relay algorithms, though, are often disabled, altered, or otherwise suppressed by malware.

In this paper, we first provide background on the main types of failures that may occur in an electrical power substation after relay algorithms are disabled by malware. We also provide some initial insights into malware methods that involve physics-informed data manipulations, which in turn may lead to power outages and physical damage to power transformers. We then describe the design of a watchdog algorithm that is continuously on the look out for anomalies in the execution time of relay algorithms along with their associated performance counters. We implemented the watchdog approach in Python, and evaluated it empirically on emulations of differential and harmonic protection algorithms on a computing machine.

1. Introduction

Industrial control systems (ICS) are used in many industries like power generation and distribution, water treatment, chemical manufacturing, oil and gas systems, and even in nuclear power plants and automobile factories. Depending on the industry, each ICS operates and manages or automates tasks differently.

Since these ICS systems and networks usually serve critical infrastructures, then getting them connected remotely may open the door for attackers to target these systems. Unlike traditional IT systems, failure in an ICS may cause significant physical damage to the infrastructure and its impact is high and the involved risks are critical. This is especially true when a power substation is targeted with a sophisticated attack, which may cause power outages and damage to substations and transformers [1].

Our focus in this paper is on protection algorithms in power substations under the International Electrotechnical Commission standard 61850 (IEC61850), since it is common in digital substations [2].

In regular IT systems, confidentiality is more important than integrity, but in an ICS, the integrity of data is considered more important and any falsification of data or injection of a malicious command targeting the physical process will lead to catastrophic results [3].

In contrast, when we spotlight ICS network protocols, unfortunately, traditional IEC61850 and the Generic Object Oriented Substation Event (GOOSE) protocols lack the needed authentication and confidentiality

requirements by design. Therefore, a wide attack surface remains open for those intruders targeting systems implemented using these protocols in case the enhanced IEC62351 was not employed [4].

IEC launched the IEC62351 standard, which is designed for achieving security in energy management systems aimed at including the authentication of data transfer through digital signatures, ensuring only authenticated access, and guaranteeing prevention of eavesdropping and spoofing [4]. Many protocol related stacks have been enhanced and many enhancements were added so intrusion detection systems can identify important properties for messages.

However, the IEC62351-6 standard outlined the security protocols for the samples measured values messages (SMV's), stating "applications using SMV and requiring 3 ms response times, multi-cast configurations, and low CPU overhead, encryption is not recommended" [5]. So in this case, SMV messages are left without encryption and spoofing can be a possible threat.

Given this, our goal is focused on mitigating physics-based attacks in the industrial smart grid. More specifically, we interested in power transformers because they are the most essential and most expensive units in a power substation.

In this paper, we proposed a new hybrid approach, which is a watchdog model to detect deviations of protection algorithms for a standard protection relay in a power substation protected zone. We

* Corresponding author.

E-mail address: haltarazi@Oakland.edu (H. Tarazi).

<https://doi.org/10.1016/j.ijcip.2024.100660>

Received 20 April 2023; Received in revised form 8 December 2023; Accepted 13 January 2024

Available online 17 January 2024

1874-5482/© 2024 Elsevier B.V. All rights reserved.

Table 1
Parameters used in a differential protection algorithm.

Parameter name	Description
Principal current I_p	Current entering the protected zone
Secondary current I_s	Current leaving the protected zone
Input voltage	Voltage to be transformed
Output voltage	Transformed voltage

developed and assigned run-time breakpoints at the beginning and the end of a typical power cycle while the protection algorithms were running in a simulated environment in their optimal operation mode. The breakpoints aimed to capture and measure the performance counters. Furthermore, we compared the consumed performance counters when the protection functions were called.

Multiple metrics can directly or indirectly relate to malware impact on the system or the running program. Performance counters are data metrics indicators designed to monitor system resources. The parameters of the performance counter are related to machine-wide resource utilization such as processor time, page fault, memory utilization, network and disk memory.

We collect performance counters of the protection algorithms when the signals are triggered. Signals are generated and formed by measuring primary and secondary current magnitudes, voltages and current ratios when the algorithms are simulated in place. In our previous work, we designed and implemented the algorithms [6].

Attackers can inject malicious control messages to alter protection measurements and differential protection algorithms leading to failure conditions in power substations. Differential protection is the main scheme used for protecting transformers from current and voltage sudden or unexpected changes in current and voltage. It works on the principles of Kirchhoff's laws for voltage and current (KCL and KVL), as well as calculating the differential ratio for primary and secondary currents for detecting faults and short circuits.

Kirchhoff law states that the total sum of differences in the differential current flowing into a protected zone is zero. In case an internal fault is detected by the differential relay or an excessive temperature is reached and passed the thermal protection limit then the relay will trip the circuit and protect the transformer. Moreover, if the differential current or current delta is verified and no fault was present in the transformer, the circuit will not be affected. This is the basic working principle of a transformer differential relay.

Our designed model is based on the Siemens SEL-487E protection relay which provides protection for transformer applications [7]. We monitor a list of metrics and calibration parameters per cycle for each protected zone. As shown in Table 1, our protection algorithm parameters are: input or principle current (operating current magnitude) I_p , output or secondary (restraint current magnitude) current I_s , and the corresponding voltage magnitudes.

As shown in Fig. 1, these readings are measured through the merging units which act as an interface between the substation bus and the process bus by mapping the internal objects to the presentation layer and the ethernet link layer based on the sampled measured values protocol (SMV's) [1].

In a summary, our proposed model aims to:

- Minimize and limit unexpected faults in the differential protection algorithms on each power cycle by detecting timing anomalies.
- Capture and compare the consumed performance counters on the first program pause (1st timestamp), and the second pause (2nd timestamp), which will be discussed thoroughly later in this section and the evaluation section.
- Build an anomaly indicator based on faulty execution time deviations and the corresponding consumed performance counters.

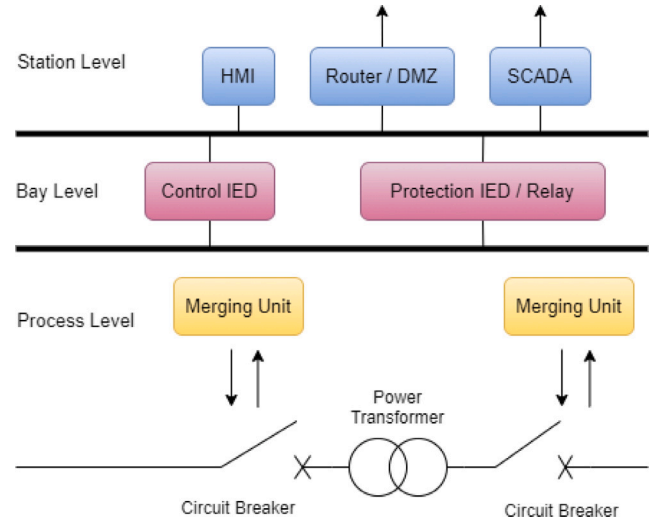


Fig. 1. IEC61850 based substation architecture.

The samples are collected after the first program pause which is intended to capture the first sample of execution time and performance counters, after the program resumes and the second pause comes in place. When we compare execution time on the first program pause, we read execution times before calling protection functions and reading from the buffers. After the program resumes and the second pause or break comes, we can collect execution times after the protection functions were called and the protection algorithms were activated.

1.1. Novelty and contribution

Our vision of the proposed model to detect hidden malware and possible anomalies is based on the exploration of the inner workings of a power transformer and protection algorithms; more specifically, differential protection algorithms in protection relays.

Few research papers addressed anomaly and stealth malware detection based on execution time and resource analysis, this work achieves two milestones. First, advancing the literature with a model to detect internal physics-based cyber attacks and uncovering the ways such attacks can target differential protection algorithms by emulating a differential physics-centric malware attack.

Secondly, it provides a method to assess protection algorithms' safety and correctness based on different factors, such as anomaly-based malware detection.

Our proposed work is among the first approaches that provide a realistic assessment of protection algorithms that consume resources, such as memory and execution time, together with a sample of physics-centric cyber attacks on power grid equipment. Our work emphasizes the evaluation of how the detection algorithm will react and the possibility of activating leveraged attacks to model better defense from current and future malware.

We emulated a differential attack that can be activated without the risk of damaging physical equipment. This allows for an unlimited number of experiments that can be customized and modified to test any factors of interest, cyber or physical, without concerns.

This research is linked to some other important studies as well. Some real-world malware's have physics-centric modules, so our research addresses a realistic problem. For instance, according to Mazepa et al. [8] BlackEnergy was originally a distributed denial of service (DDoS) malware, but was later used as a part of an attack that targeted power substations in eastern Europe in 2015. The malware, once activated on information technology workstations, allowed attackers to gain access to the control system and directly control the physical processes of the substation and cause power distribution halts.

Our attack methods differ from BlackEnergy in that they attempt to emulate how physics-centric modules could cause physical damage to power grid equipment, while [9] found that BlackEnergy primarily affected power distribution.

As far as we know, the Aurora Generator Test [10] was the first to demonstrate how physics-centric cyber attacks can cause physical damage. Our target is to continue Aurora's line of work but from a defender point of view, where we propose a model to detect anomalies and hidden physics-centric differential malware. This is done through emulation to avoid destroying actual power grid equipment while maintaining a high level of integrity for research goals.

2. Background and research problem

In the past, power control systems in general were designed to be isolated physically and locally connected to the industrial base. Recently, with the rise of the internet connectivity era, these systems can be accessed remotely or through a special or custom VPN or (Demilitarized zone) DMZ connection depending on their network architecture. Therefore, securing communication channels for these systems and between control servers and the industrial field equipments is critical.

The architecture and network topology of an ICS is complicated, and there are a number of special communication protocols for industrial automation and control systems, like Modbus, Dnp3, GOOSE, MMS, Mirrored bits, Profinet, Profibus, and S7comm, depending on the manufacturer.

In this paper, we focus on IEC61850 because it is unique in its capabilities and it offers interoperability between various manufacturers. IEC61850 provides a unified model for how power system devices can organize data in a manner that is consistent across all types and brands [1]. This eliminates much of the non-power system configuration efforts because IED devices can configure themselves [1].

For instance, if one connects an IEC61850 relay, the system can detect it automatically and map its calibrations to the main control workstation system without user interaction. Some devices use an SCL file to configure the objects and engineers only need to import the SCL file into the device to configure it. Then, a client application can extract the object definitions from the device over the network [1]. This reduces the costs and efforts needed to configure an IEC61850 device.

For example, control messages containing sampled measured values (SMV's) flow from the SCADA control workstation to the protection relay, then the relay communicates with the connected field devices or Intelligent Electronic Devices (IED's) and merging units, as illustrated in Fig. 2 above.

A few prior research projects and papers addressed predicting and detecting abnormalities and anomalies of protection algorithms. Production and power engineers design these algorithms in the testing and modeling phase, but in reality, it is highly recommended to assess the algorithms during operation.

In accordance with Kirchhoff's law, the difference between the primary and secondary currents should be always zero. Therefore, any unexpected differential current reading should be detected by the algorithm by default. This difference is already calculated by the differential protection algorithm and the total execution time for each power cycle should be calculated periodically and should remain around a threshold value.

Power transformers are designed and built to last for a long time, and as we mentioned earlier, the costs and efforts to build and maintain a power transformer are high. Therefore, many safety and protection measures and techniques are employed to protect them, covering all kinds of faults and failures and considering natural risk factors like floods, storms, earthquakes, fires, and many other external factors.

While protecting power transformers from internal faults like short circuits and over currents is complicated, since there are some dependencies and overlaps between internal components. Still, there is a

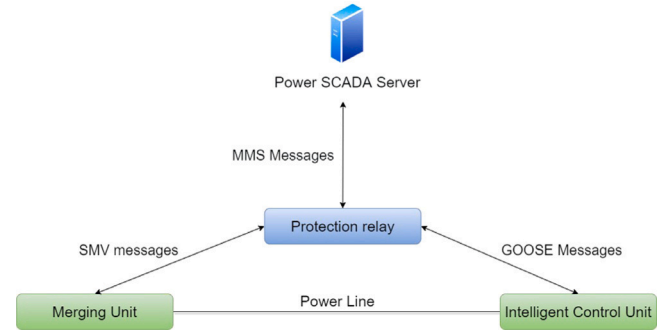


Fig. 2. Control messages flow in a protected zone.

well detailed and sophisticated protection mechanism for each kind of failure, both mechanically and digitally by digital protection relays.

We can simplify the states of a power transformer as follows: first, the transformer is energized, then after a transient state for a specific period of time, the transformer will normally run in its steady state (as seen in Fig. 6). But in case of a failure, the transformer will turn into the failure mode state, and generally, there are four common types of failures in power transformers, namely:

- Over Currents
- Short Circuits
- Tap Changer Failures
- Over temperature and thermal protection or cooling failures

2.1. Transformer differential protection

The differential protection (ΔI) principle is based on comparing the output and input transformer currents, and power transformers operate with the nominal current. Current transformers (CT) are selected with a corresponding turns ratio where currents in CT secondary sides are equal [11].

In this case, there is no current flowing when $\Delta I=0$ because secondary currents have equal amplitude and phase displacement value, so that (ΔI) will not operate [11].

In faulty conditions, the transformer current value will be higher than the nominal current which will cause (ΔI)>0. In this case, protection functions will operate and send a tripping signal.

2.2. Harmonic restraint protection

Harmonic protection works when the circuit is being energized and an inrush current is started. The inrush current has harmonic waves that will distort the original current wave and its amplitude [6]. This negatively impacts the differential protection algorithm causing altered differential current ratios being rated. Thus, the overall protection algorithms outcomes will be distorted.

2.3. Research problem

The proposed watchdog model aims to minimize fault latency for protection algorithms on a power cycle which causes an unexpected tripping signal to be sent. This may yield a power outage, or when an unexpected circuit breaker status is noticed, it causes improper feedback to the protection algorithms logic, which may lead to damage to the transformer components in the long term. Based on the specifications of the relay, we assumed that the target fault resistance time is less than 3 s, in case the internal fault latency in the protected zone remained less than 3 s this can be considered safe for the power transformer [7].

By default, protection function calls are triggered when an over-current situation occurs, or when the differential protection algorithm detects a difference in the differential current ratio. We added program pauses in the implementation of the algorithm. Therefore, we relied on recording and tracing execution times and the consumed performance counters using specific PowerShell and python scripts intended to run before protection algorithms are launched and triggered.

After collecting execution times per single state of the power transformer, execution intervals may vary in their lengths while the threshold value will remain as the center point, some intervals have longer lengths, and some others have shorter lengths, the overlapping intervals fluctuate within that time frame, we aim to detect those intervals exceeding the limits.

Typically, protection algorithms utilize a minimal amount of memory on disk and minimally access memory on secondary storage. Therefore, they trace execution times, memory consumption, and network utilization out of a protection algorithm can help in detecting malicious malware behavior. This is because malware consumes more memory resources than the original amount of memory the protection algorithms and the program is utilize.

Malware usually makes use of secondary storage and network interfaces to establish a covert channel and infect a system. Furthermore, it tries to conceal its behavior by acting as a legitimate program or service, while in reality, it targets sensitive data or other programs in the system. Thus, whenever a malware accesses secondary storage or utilizes memory on disk, the changes will impact the amount of performance counters consumed. This occurrence indicates that the system or the algorithm has been compromised.

In this research paper, we do not cover network traffic anomaly and malware analysis. Indicators of malware infection and anomalies are discussed and studied locally in the system; we will cover more aspects in our future research as a second phase.

2.4. Attack methods

Intruders usually target electrical power grid networks, workstations, and field devices because these systems are considered a high-value target and the impact of successful attacks is high since these systems serve critical infrastructures, and if they were interrupted or attacked, it may result in a dramatic financial loss and threat to human lives, as mentioned earlier.

ICS systems can be exposed to a broad range of cyber attacks ranging from remote attacks to network and application-level attacks, and even hardware-level attacks like firmware modification. From another point of view, according to cybersecurity experts and the InfoSec institute, an unpatched vulnerability might be the root cause of the most successful attacks against ICS systems, and also it is difficult to mitigate and defend against attacks in critical infrastructures in real-time operation [12].

According to an interesting report published by InfoSec for the second half of 2020 [12], critical and manufacturing infrastructures were the most impacted industries by ICS vulnerabilities. As illustrated in Fig. 3 above, 194 vulnerabilities were discovered in the critical manufacturing sector, which means that securing these systems is really a high priority for governments and top tier companies.

According to another report published by the industrial cybersecurity firm Claroty [13] providing statistical data for the second half of 2020, the number of discovered vulnerabilities in industrial control systems in 2020 was 24,72% higher compared to 2019. The researchers announced that 449 vulnerabilities were disclosed affecting ICS products from 59 vendors during the second half of 2020 [13]. This is a shred of clear evidence the situation is serious, especially with the fact that over 70% of the reported issues were rated as high or critical based on the common vulnerability scoring system (CVSS).

We narrowed down our scope of threats to attacks targeting electrical power transformers in the electrical power grid, more specifically

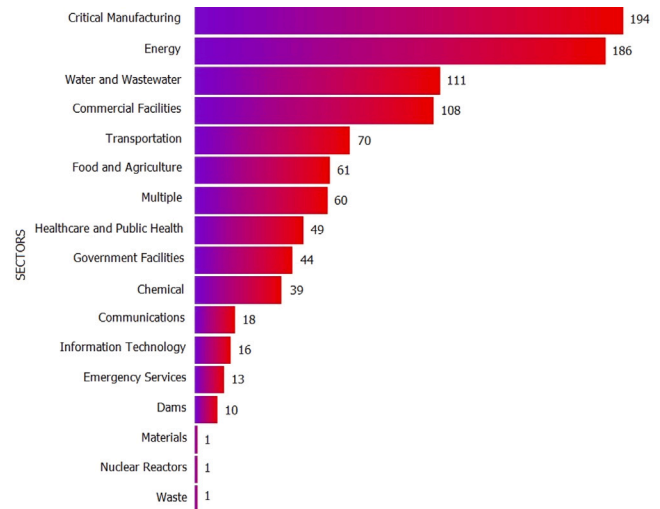


Fig. 3. Reported vulnerabilities for the second half of 2020.

Source: The above figure was reprinted from ICS/SCADA threats and threat actors, a Journal in Infosec Critical infrastructure, Retrieved September 14, 2021, from <https://resources.infosecinstitute.com/topic/ics-scada-threats-and-threat-actors>. Reprinted with permission.

© 2021 by Pierluigi Paganini.

attacks and threat actors targeting protection algorithms. Malware and attacks with crafted malicious SMV messages can run harmful codes on industrial computers which may affect the protection algorithms, the malicious codes may access buffers storing current measurements and physics data used by the protection algorithms [14].

As an example, one of the ways in which malware could cause physical damage to a power transformer consists of affecting the protection algorithms on a compromised industrial computer. When a fatal fault occurs, the compromised industrial computer fails to trip the circuit breaker. This behavior is observed in some malware's such as Industroyer [15] and Triton [16]. For example, if malware was able to trace harmonics, and if any harmonics indicate a fault, malware may prevent circuit breakers from tripping, which in this case can lead to physical damage to the power transformer. In conclusion, the concepts explained in this section are the central physics that we refer to in this paper. Additional physics are referenced and explained in other sections of the paper as needed.

In [6], the authors presented a framework of three attack methods described as the physics-aware attack method, the interposition attack method, and the coordinated attack method. These three attack methods work together to compromise the system.

A physics-aware attack is an attack wherein the threat actor utilizes the knowledge of the physics of the ecosystem to launch an attack against a cyber physical system [6]. A cyber physical system is coupled physically with the ecosystem to monitor system behavior through inputs and control system behavior through various outputs based on the function of its control algorithms. This attack will identify **sensitive data** [6] that it will manipulate to deceive the cyber-physical system to compromise the system and exert control over it. Per [6], this **sensitive data** could be system inputs, temporary calculations in memory, or system outputs. Since we are dealing with a computer system, these are the variables stored in memory that are being altered in some way by the threat actor. This alteration uses the physics knowledge of the ecosystem to provide meaningful values to the system and cause its algorithms, which rely upon the laws of physics, to respond in ways the threat actor desires, thereby altering the normal behavior of the system.

An interposition attack is an attack wherein the threat actor interposes the flow of the system to stand in between a function's input

and its output and alters a value as described in [17]. In [6], the authors extend this concept into cyber physical systems. In a control system network, this is typically sensor data that is manipulated in some way with playback to give artificial results that manipulate the control algorithms. This results in deception within the system that will manipulate the system. When coupled with a physics-aware attack, that manipulation allows the threat actor to control the overall system behavior.

In [6], the authors describe a coordinated distributed attack such that the threat actor will execute various attacks to compromise targeted subsystems in a planned and coordinated attack to gain control of each subsystem and then in turn manipulate the overall system. This coordinated attack is many times required in a control system as they are made up of subsystems that control and monitor various subsystems that work in concert with the overall system. Just impacting one subsystem might not be enough to exert the control desired. For example, a threat actor might need to overload the current in one subsystem by altering sensor readings while in another subsystem suppressing warnings and protection systems that detect and deal with operational and safety anomalies for operators. The result is an undetected fault that leads to damage to the overall system when the protection systems do not operate as they are deceived and see no faults.

When the three attack methods are used in concert with each other, a threat actor can completely control the system's behavior. In control systems, this typically results in physical manifestations that can damage the system or harm others nearby. For this reason, it is critical to find defense methods to mitigate these attacks.

Substation network communication uses cryptographic protection for authentication, so spoofing attacks on the wire raw are not possible. Once an exploit penetrates an industrial computer and hence brings malware modules on it, the malware code may have access to cryptographic keys and digital signatures, which can be used to send malicious network traffic to other industrial computers on behalf of the compromised machine.

In this way, a spoofing attack can be possible because the malicious network communication is generated by malware code rather than the actual protection algorithms. Nevertheless, from a cryptographic perspective, those network communications appear to be valid because of the use of valid cryptographic keys.

Physics-aware attacks originally aim to cause damage to the electrical substation and hence disrupt the transmission or distribution of power. Protection and safety measures can be also targeted. Meanwhile, an advanced attack can be designed and modeled by an expert in the field of power substations, and the experienced attacker can inject some malicious codes or crafted messages that can alter some configurations or stored instructions within a control system master station, or a field device. Moreover, an attacker can exploit a vulnerability in the operating system running the control system software itself and implant the malicious code or script, run or stop a specific service, and even he or she can open a backdoor or spoof network traffic.

IEC61850 protocol based automation and control systems have the advantage of protection, while control systems based on legacy protocols in the past suffered from disorganization, and modeling of how data should be undefined in terms of the application and mapping. So power systems engineers used to manually configure objects and map them to power system variables and low-level register numbers, (Input, Output) I/O modules, index numbers, so this issue opened an extensive interface or playground for internal attacks or social engineering attacks to take over.

As we see in Fig. 4 above, a spoofing attack for example, can target a protection relay with a spoofed SMV message containing altered contents and instructions through the network after sniffing the original messages and manipulating them.

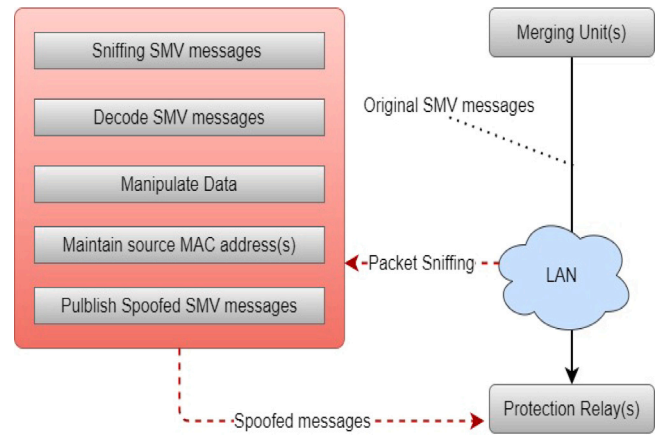


Fig. 4. Example of a Spoofing attack for SMV messages.

3. Threat model

We describe our threat model by acknowledging the types of motivations that typically drive threat actors who compromise industrial control systems. Industrial networks do not tend to hold information that an intruder would find helpful or profitable. No passwords or other sensitive information can be found on a plant's PLC. Instead, we have to assume that an attacker who seeks to enter into a CPS controlled system does so to compromise the safe operation of the assets and cause physical damage to the plant and potential personnel. Thus, our threat actor does not spend time looking for traditional sensitive data but rather seeks data related to the physical operation of equipment, see Fig. 5.

Our assumptions of a threat actor who can gain entry into a CPS are as follows. The threat actor can locate a PLC on the industrial network, install malware on it, and run arbitrary code. This includes viewing physics data as it enters the logic controller and modifies it before or after it is processed by protective algorithms running on the device. The malware may inspect any section of memory, including data held in registers.

This physics determines the decision reached by the differential protection algorithm. In other words, knowledge of the location in memory, the range of acceptable and unacceptable values, and algorithmic logic allows a threat actor to cause a circuit breaker to trip when there is no fault or to suppress a trip when a genuine fault has occurred. Through this manipulation, a threat actor may force a device in an industrial network to operate in an unsafe state, potentially leading to a catastrophic failure.

3.1. Malware attack emulation

As described earlier, the differential protection algorithm works in the heart of the digital relay by calculating the ratio of the primary and secondary currents stored in the microcontroller memory, if the sum bypassed a threshold stored in the memory, a trip signal will be issues to the circuit breakers to isolate the power transformer.

The malware attack on the protection algorithm have the goal of controlling when the circuit breaker will be tripped, or suppresses a trip completely as it tries to deceive the control logic of the protection relay by manipulating the circuit breaker boolean variable which indicates that the breaker is tripped or not by having a reference in the memory location of this variable. The malware will identify changes in this value and send an altered value before the real signal is issued to circuit breakers.

Moreover, depending on the physics data available, the malware will overwrite the restraining current value or excess the value stored

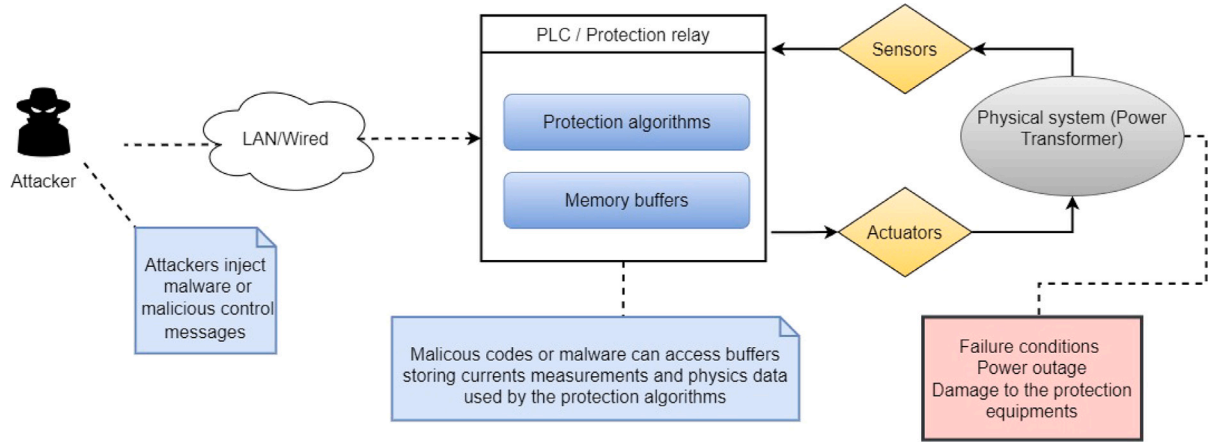


Fig. 5. Threat model.

the differential current buffer with the aim to alter the calculated ratio and causing a false fault and issue an unnecessary tripping signal to circuit breakers repeatedly.

These scenarios are emulated in the experiment where the malware can be activated while the protection algorithms are running normally, or after activating a noise function. Afterwards, the impacted performance counters and the noticed memory consumption are tracked along the timeline.

4. Watchdog approach

In this section, we explain our watchdog detection model and algorithm for detecting anomalies based on four factors which are:

- Tracing and comparing execution times at each state of the power transformer
- Identifying and comparing the consumed performance counters on each state
- Identifying and tracing disk memory consumption on each time stamp

As shown in Fig. 6 we categorize the states of the transformer as follows:

- Normal steady state
- Transient state
- Off
- Failure modes:
 - Over current
 - Short circuit
 - Tap changer failure
 - Over temperature and thermal protection or cooling failures

4.1. Tracing and comparing execution times at each state

We characterize the duration in time of each cycle of the protection algorithms as a function of the power transformer state. Therefore, we trace the execution time of protection algorithms over program flow with relation to a specific state to the power transformer, then identify odd execution times that exceed normal limits (which is identified by a threshold, and two time stamps).

In our experiment, we focus on measuring the consumed memory on disk or secondary storage periodically and on each time stamp in order to uncover any unexpected memory consumption by the protection algorithm. Usually, when a hidden malware operates in the background it consumes more storage memory than expected by the algorithm.

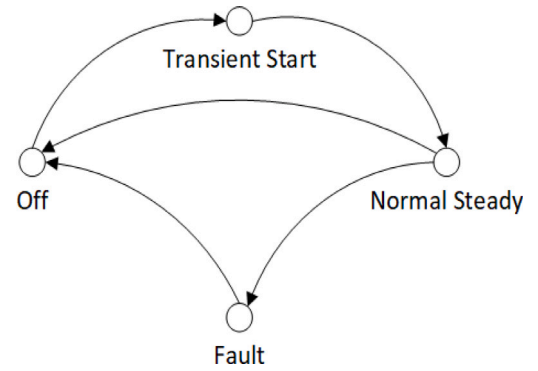


Fig. 6. State transitions in a power transformer as seen by an industrial computer.

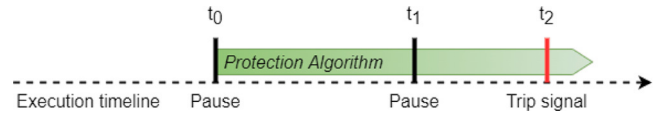


Fig. 7. Execution time line with time stamps.

When the first time stamp (t_0) is recorded, execution time right before reading from memory and calling the protection algorithms will be measured. After resuming the program, the second timestamp (t_1) will retrieve the execution time, but during algorithm's operation, we will then proceed to compare execution times on different time stamps, since it provides time measurements for normal versus abnormal execution states for the protection algorithms, see Fig. 7

4.2. Identifying and comparing the consumed performance counters on each state

This factor is obtained by exploring the types and amounts of the consumed performance counters within the same time interval and timestamps. After performance counters are collected, the scenarios or states (e.g., possible false positive or false negative cases for a transformer relay) are used to decide whether to trip the circuit breaker or not, and can be summarized as follows:

- The program started the protection functions without reason or without a decision from the differential algorithm since the differential current is measured and found normal, and this comes in parallel with the fact that the malware aims to change the restraining current, assuming that no faults were present in the relay, and this is the false positive case.

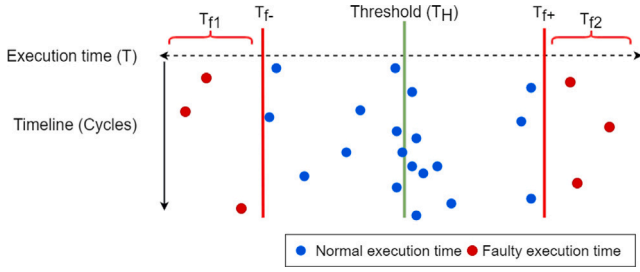


Fig. 8. Execution time intervals around the threshold.

- Protection functions were not triggered even when a decision was made by the differential algorithm after doing its calculations, and this is considered a false negative case.
- Another possible false positive case happens when a high-rated inrush current (up to 100 times the steady state current) flows during the transient state and decays after a long period of time [18]. This can impact the differential ratio, and hence the protection algorithms will read abnormal current ratios which causes an unexpected tripping signal to circuit breakers.

Our approach does not cover cases where the differential current ratio is unavailable due to sensor malfunctions or physical damage. We excluded thermal protection functions and internal physical faults from this work.

4.3. Watchdog components

Our detection model consists of the following aspects:

4.3.1. Execution timeline

We measure and characterize execution times for the protection algorithms per timestamp as a function of the power transformer state. We have a different execution time collected for each power cycle. If we consider (T_{f1}) , (T_{f2}) to be the intervals with faulty execution time, we have (T_{f+}) as an upper limit with a shift difference longer than the threshold (T_H) , hence (T_{f+}) is the interval within boundaries (T_H, T_{f+}) , and (T_{f-}) is the shift difference shorter than T_H , with interval boundaries (T_H, T_{f-}) . Boundaries (T_{f+}, T_{f-}) are the min and max limits of normal execution time (after considering some factors that may slightly impact execution time like network latency, noisy signals, etc.) , while (T_{f+}, T_{f1}) or (T_{f-}, T_{f2}) are the execution times for anomalous execution times, as illustrated above in Fig. 8:

Therefore, Eq. (1) is assigned for calculating (T_{f2}) :

$$T_{f2} = (T - T_H), \quad \text{if } T > T_{f+} \quad (1)$$

And Eq. (2) stands for calculating (T_{f1}) :

$$T_{f1} = (T_H - T), \quad \text{if } T < T_{f-} \quad (2)$$

At any time if we get a value out of limits, we can find the shift differences by comparing them to the minimum and maximum limits, as expressed in algorithm 1.

4.3.2. Types and amount of the consumed performance counters

Multiple categories of performance counters can be obtained through windows performance monitor (*Perfmon*) using a PowerShell script, the script scope is intended to capture the counters for the currently running process in the emulated program before and after the breakpoints with a continuous sample interval. We focus on execution time, memory page faults and the consumed memory on disk (secondary storage). See Table 2.

Algorithm 1: Finding execution time deviations and performance counters

Input : T, T_H, T_{f+}, T_{f-}
Output: Execution time deviations: T_{f1}, T_{f2}, PC_t

```

1  $T$  : Execution time;
2  $T_{f2}$  : Faulty execution time  $> T_{f+}$  ;
3  $T_{f1}$  : Faulty execution time  $< T_{f-}$  ;
4  $T_H$ : Threshold limit value;
5  $PC_t$ : Performance counters per timestamp;
6 foreach Timestamp  $t$  do
7   Function Detect_faulty_execution_time( ):
8     if  $T > T_{f+}$  then
9        $T_{f2} \leftarrow T - T_H$ ;
10    return  $T_{f2}$ ;
11    else if  $T < T_{f-}$  then
12       $T_{f1} \leftarrow T_H - T$ ;
13    return  $T_{f1}$ ;
14  Function Detect_performance_counters( ):
15    Record  $PC_1, PC_2, PC_3$ 
16    return Performance counters per  $t$ ;
17  return  $T_{f2}, T_{f1}, PC_t$  Per  $t$  (If available)

```

Table 2

Performance counters used in the experiment.

Performance counter	Description
Execution time	Execution time required in milliseconds
Memory page faults	Page faults per sample
Disk memory	Memory consumption by the algorithms

4.4. General approach algorithm

Our detection methodology is summarized below in algorithm 2, where deviations over protection algorithms are detected after collecting and comparing different indicators as we mentioned earlier; First, the algorithm will read values obtained for the first time stamp t_0 from the first algorithm functions, which are faulty execution time and recorded performance counters, notice again that protection algorithms are not triggered yet.

Then the algorithm will again read values for the second time stamp t_1 . Here the difference between execution times is compared in addition to the consumed performance counters and their amounts during this time interval. In case the execution time found increased, then this is a sign that protection functions were started, and it is essential to note that the recorded execution time should remain within the min and max limits that we specified earlier.

However, if the execution time difference was found to decrease and exceeded the lower limit, then this is the first kind of anomaly to be recorded because this is a sign that protection algorithm codes are bypassed or did not operate properly.

Secondly, two types of performance counters for the same interval are analyzed and compared here, PC_1 (memory page faults) and PC_2 (memory on disk or secondary storage), memory page faults are compared. They need to be semi-equal on both time stamps, while consumed memory on disk needs to be traced along the execution timeline continuously based on a 1-second sample rate.

As we mentioned, malware usually requires secondary storage and access to the file system. At the same time, protection algorithms mainly depend on main memory, so it is important to report any sudden increase in disk memory consumption by the algorithms since secondary memory utilization should remain minimal, and this is a third anomaly indicator to notice.

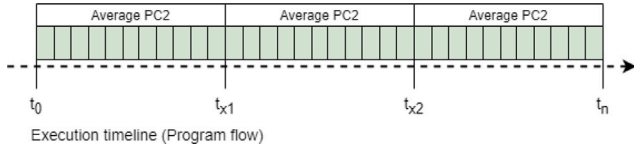


Fig. 9. Average for performance counters type 2.

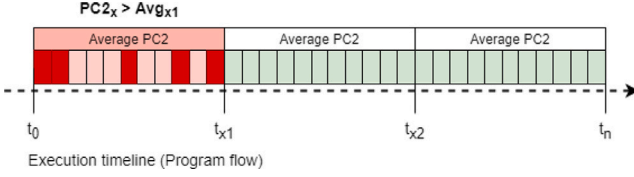


Fig. 10. Average for performance counters type 2.

A sliding window is assigned to record PC_2 starting from t_0 to t_n or the second of time in the program flow timeline, for each interval t_n to t_x , PC_2 is recorded and stored in the sliding window. The statistical average for PC_2 is calculated and compared with the statistical average for the next sliding window, as illustrated in Fig. 9.

In case the reading of any PC_2 value is found to be bigger than the statistical average, which seemed to be minimal for the protection algorithm, then this is another type of anomaly to be recorded, see Fig. 10. Finally, if the value of PC_2 was not available or if the statistical average was different on the next sliding window, then this is another kind of anomaly to be recorded, identified by the timestamp number in the sliding window.

5. Testing and evaluation

For evaluating the proposed model, as stated earlier, we relied on a PowerShell script together with the *memory profiler* python package by Fabian Pedregosa [19] for collecting and comparing the consumed performance counters before the first program pause and after triggering the protection algorithm. The targeted parameters are: Execution times, memory page faults and secondary memory consumption or memory on disk.

We started by selecting samples for normal and faulty states, then after activating the differential malware, we obtained the third sample. As the timer began, based on a 1-second interval sample rate, the execution time remained fixed at around 0.3 ms under normal conditions without faults on the first and second program pauses, as illustrated in Table 3 and Fig. 11.

Under the same conditions, page faults per second remained close to zero, and memory consumption by the algorithms was fixed at around 3.39 MB, see Figs. 12 and 13.

After the second program pause, when a noise factor of 0.1 was added to the primary and secondary signals, we noticed an increase in execution times and disk memory consumption, and the number of memory page faults increased for the same sample rate, as seen in Table 4 and (Figs. 11, 12, 14).

As a second phase, we ran the differential malware and found that execution time and memory page faults dramatically increased. Also memory usage increased after the pause to 60.2 MB, and this is a clear sign that malware will consume memory more than the memory consumed by the algorithms. This is illustrated in Table 5 and (Figs. 15, 16).

Now, suppose we assume that the threshold value equals 0.3 ms, which is the normal state execution time. In that case, we notice that execution times have different shift differences on each time interval (Fig. 17), and after the first program pause, the first three faulty execution intervals under noisy signal were above the threshold and

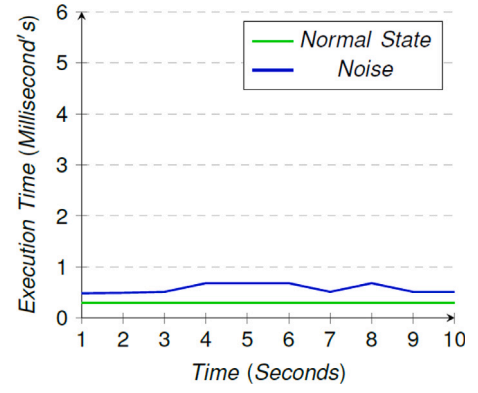


Fig. 11. Execution time after adding noise.

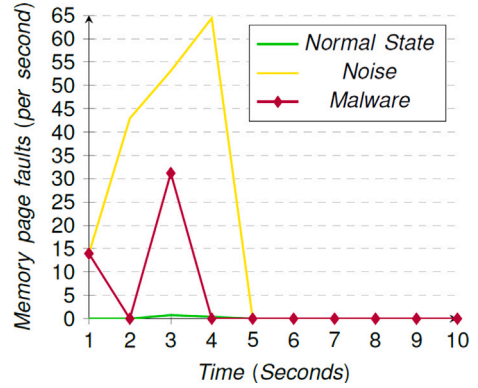


Fig. 12. Page faults per second on each state.

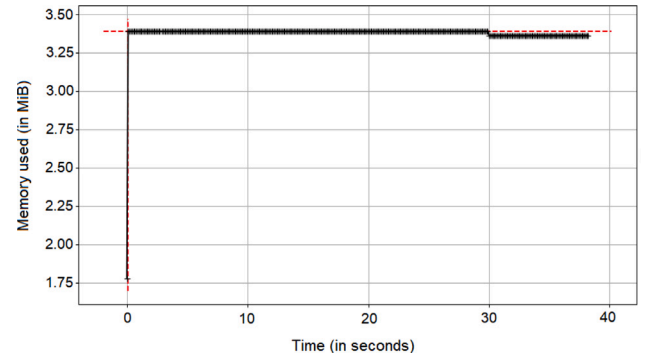


Fig. 13. Memory consumption profile at normal execution.

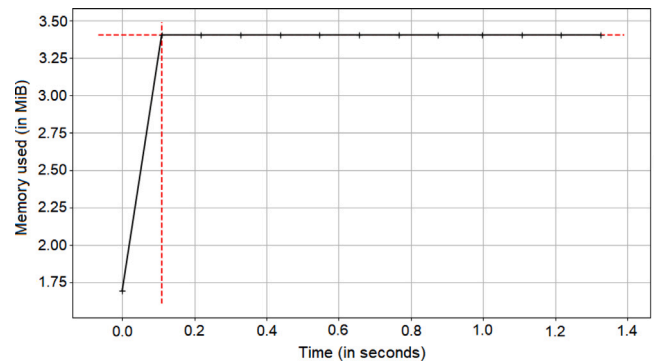


Fig. 14. Memory consumption after resuming the program with added noise factor.

Algorithm 2: General approach algorithm

Input : $T_{f1}, T_{f2}, PC_1, PC_2$
Output: Anomalies over protection algorithms

```

1  $t$ : time stamp
2  $PC_1, PC_2$ : Performance counters
3  $t_0, t_1, t_2$ : First, second and third time stamps
4 Avg: Statistical average for  $PC_2$ ;
5 TL: Sliding window:  $PC_2[t_0 \dots t_n]$  for  $PC_2$ ;
6 foreach State  $s \in S_n, t \in \{t_n\}$  do
7   Compare execution times:
8   if  $\exists (T < T_{f1}) \vee (T > T_{f2})$  then
9     Record anomaly for state  $s$ , timestamp  $t$ 
10  Compare  $PC_1$ :
11  if  $\exists (t_0:PC_1 > t_1:PC_1) \vee (t_1:PC_1 > t_2:PC_1)$  then
12    Record anomaly for state  $s$ , timestamp  $t$ 
13  if  $\exists (t_1:PC_1 \neq t_2:PC_1)$  then
14    Record anomaly for state  $s$ , timestamp  $t$ 
15 Trace  $PC_2$ :
16 ( $t$  rate = 1 second)
17 while Program is running do
18   Read  $PC_2$ 
19   repeat
20     Record  $PC_2$  in TL
21     Read next  $PC_2$ 
22   until  $t=t_n$ 
23   Calculate Avg
24   if  $PC_2:t_n > Avg$  then
25     Record anomaly for state  $s$ , timestamp  $t_n$ 
26   else
27     Next TL[n]
28   if  $\exists (t_0:t_n, PC_2 \equiv Null)$  then
29     Record TL[n]
30     Record anomaly for state  $s$ , timestamp  $t_n$ 
31   Next Timestamp

```

Table 3

Execution times and performance counters at normal state.

Timestamp	Execution time ^a	Memory page faults ^b	Disk memory ^b
1	0.3	0	3.39
2	0.3	0	3.39
3	0.3	0.78	3.39
4	0.3	0.41	3.39
.	.	.	.
t_n	T	PC_1	PC_2

^a In Milliseconds.^b Per timestamp.

remained in place after resuming the program. Hence, the detection model will record these intervals as anomalies (Fig. 18).

The same goes for execution intervals and average consumed memory after activating the malware, some intervals were far beyond the threshold limit, and some others were beneath the threshold. Thus, all these faulty execution deviations will be recorded as anomalies, as illustrated in Fig. 18.

6. Related work

Detecting attacks from physics-based perspective has been recently catching attention in cyber physical systems. Many researchers spotlighted attention on this kind of attack because it may target critical

Table 4

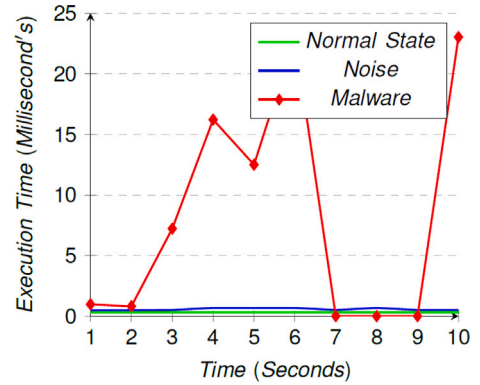
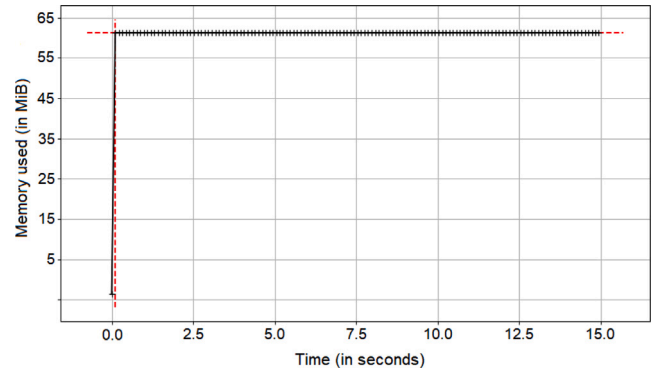
Execution times and performance counters after adding noise to the signals.

Timestamp	Execution time ^a	Memory page faults ^b	Disk memory ^b
2 nd pause	.	.	.
1	0.488	13.95	3.49
2	0.491	42.94	3.59
3	0.51	53.12	3.59
4	0.68	64.42	3.59
.	.	.	.
t_n	T	PC_1	PC_2

^a In Milliseconds.^b Per timestamp.**Table 5**

Execution times and performance counters after activating the differential malware.

Cycle	Execution time ^a	Memory page faults ^b	Disk memory ^b
1	0.976	13.95	60.2
2	0.812	0	60.2
3	7.223	31.2	60.2
4	16.225	0	60.2
5	12.504	0	60.2
6	23.034	0	60.2
.	.	.	.
x	T	PC_1	PC_2

^a In Milliseconds.^b Per timestamp.**Fig. 15.** Execution times after activating malware.**Fig. 16.** Memory consumption after activating the malware.

infrastructures and industrial control systems at most. According to a research study [20], power interruption attacks targeting power systems have been evolving recently. Researchers stated that attackers

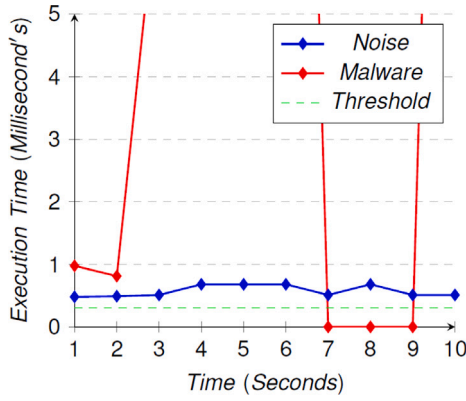


Fig. 17. Execution times after activating malware.

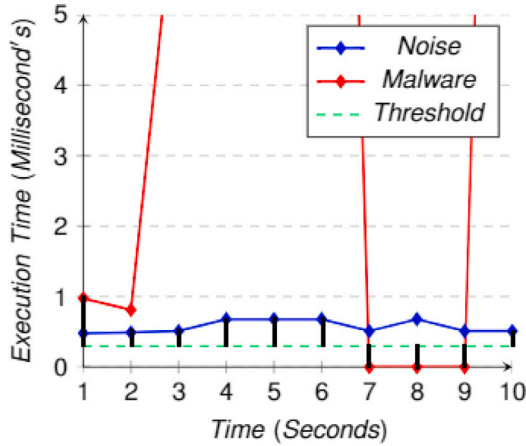


Fig. 18. Execution time shift differences around the threshold.

are able to affect the normal operation of power generation and hence they can increase losses and temper with currents and voltages causing wear and tear and aging for a specific power generator when it is overloaded and unexpectedly tripped and interrupted during peak load conditions.

Many proposed detection schemes in the literature aimed to detect false tripping and other cyber attacks targeting substations equipment in power systems, varying from behavior-based methods as in [21] to stateless and stateful traffic inspection methods as in [22] and [23]. In contrast, very few research papers studied assessing protection algorithms performance in terms of processing time and memory consumption from an operating system perspective.

According to [24], host-based malware detection based on performance counters in the context of assessing the dynamic behavior of computer programs is found effective. The authors captured 25 different operating system performance counters and stored them as time series in the presence and absence of malware. Then, they used ten different deep-learning models for time series classification. They concluded from the experiment that the performance counters accurately differentiate the dynamic behavior of programs over a given period of time. They claimed that the results were not affected by the physical or virtual environment running the program.

In this work, we did not dig into deep-learning models for time series classification as we are planning to employ it in the future paper. We focused on capturing the affected performance counters in our experiment, namely disk space, execution time, and memory page faults, and distinguished them between different states during the protection algorithm execution. We did not include the set of all performance

counters in our testing scope as it is considered time-consuming and not cost-effective.

On the other hand, anomaly-based malware detection is important in detecting malware that successfully infiltrates other defenses, stealth or hidden malware and zero day malware or attacks require new ways for detection. The presence of an anomalous behavior indicates the presence of hidden malware or rootkits, or even disguised processes. Vaas and Happa [25] presented a detection method for deviations or anomalies based on the concept of misuse intrusion detection systems where normal application behavior is profiled by observing process memory consumption.

The authors gathered memory snapshots of a target machine using *psutil* [26], which is a cross-platform library for capturing information on running processes and system utilization in Python. It allowed them to record snapshots of all processes, including their resource consumption. The result was a single time series of memory usage for each process. The authors argued that due to the execution time of *psutil*, sample acquisition was limited to an interval of 60 s.

Moreover, a snapshot can include multiple processes of the same application which they incorporated into their model as individual runs. In contrast, in our proposed model, we monitored and collected memory consumption on disk based on the *memory profiler* package [19], which allowed us to gather and plot memory usage as a function of time for specific functions or pieces of code in the program code itself precisely for an unlimited sample acquisition interval, and with the ability to monitor program or code memory usage in milliseconds.

Accountable research has been done to model attacks and provide a framework for mitigating threats against power systems, according to [10]. The Idaho National Laboratory conducted the Aurora Generator Test in 2007 to showcase how a cyberattack can destroy the physical components of the power grid. They used a computer program to rapidly trip a diesel generator circuit breakers out of phase from the rest of the grid causing it to explode. This vulnerability was known as the Aurora vulnerability.

The Aurora vulnerability remains a concern because some power equipment still depends on legacy networking protocols that were designed lacking security principles, such as authentication, confidentiality, or replay protection. As a cost effective and fast method to explore such attacks, our approach makes use of emulation to achieve goals similar to those of the Idaho National Laboratory, but from a defensive point of view. With emulation, an attack can be demonstrated without destroying equipment.

To the best of our knowledge, the Aurora Generator Test is the first to demonstrate physics-centric cyber attacks and their ability to cause physical damage. Physics-centric attacks on physical processes and equipment were also discussed in another research by Garcia et al. [27]. The authors designed a toolkit, named HARVEY, which demonstrated extensive knowledge of physics, and hence knew how to manipulate physics such as to maximize the damage to power grid equipment based on external faults [6].

By comparing this work to the work of Garcia et al. [27], our work addresses a physics-centric malware attack and its impact on protection. The difference is about physics, how physics is leveraged by the two attacks and what are the types of the resulting faults as well as how the two approaches interplay with protection algorithms on industrial computers.

In contrast to Harvey, our emulated differential attack and the emulated attacks by Olijnyk et al. [6] leveraged the inner physics of the power grid equipment to cause internal faults, where on reality, the consequences of such attacks may include physical damage and possible power outages. Garcia et al. [27] did not study attacks on protection algorithms, but we believe that HARVEY can be applied to those algorithms as well by dismissing their computation results. Attack methods by Olijnyk et al. [6] dove deeper into control algorithms, in addition to interposing between those algorithms and the I/O board, while our approach in this paper made use of attack emulation together with resource analysis to analyze attack effects.

7. Conclusion and future work

The goal of this paper was to highlight the importance of researching the concerned physical characteristics of power transformer protection relays and the underlying algorithms. We proposed and implemented a hybrid anomaly and malware detection model for differential protection algorithms. Since execution time for protection functions tends to increase or decrease when abnormal conditions occur to the protection relays, hidden malware could negatively impact the process causing unexpected disk memory consumption or reporting false results to the protection algorithms and the control system, affecting the overall safety and protection measures for power transformers.

Our model covered the possible alternative scenarios in which a differential algorithm can be deceived, and resolved these scenarios by using performance metrics. We emulated abnormal scenarios and activated and implemented a differential malware against the protection functions during run-time and evaluated its performance. Our results showed that having a multifactor approach for detection is practically feasible and effective.

However, our model is not designed to deal with all kinds of internal faults. We did not cover the possibility of having external to internal faults and thermal protection faults, but this will be covered in future research.

For future work, some researchers addressed possible ways for detecting time series anomalies with static or dynamic thresholds using convolutional neural networks (CNN) or long short term memory (LSTM) deep learning models for predicting and estimating anomalies [28]. We aim to take this paper as a core for building an advanced hybrid model for predicting anomalies using machine learning and defensive deception.

Moreover, after the execution time is measured on each timestamp of program flow, and based on heuristics' analysis for the stored signals or current values in the memory buffer, the anomaly indicator can be empowered with a machine learning algorithm to learn both the standard and abnormal behavior of protection algorithms, and distinguish between each different state of the power transformer and its expected corresponding execution time of the protection functions, and this will be discussed in the future.

Declaration of competing interest

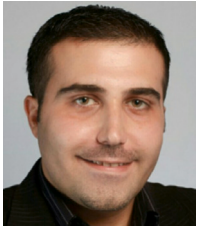
The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] M. Adamiak, D. Baigent, R. Mackiewicz, IEC 61850 communication networks and systems in substations, *Prot. Control J. Smart Grid* (2010) 61–68.
- [2] R. Gupta, Substation automation using IEC61850 standard, in: *Fifteenth National Power Systems Conference, NPSC, IIT Bombay*, 2008, pp. 462–466.
- [3] C.M. Ahmed, J. Zhou, Challenges and opportunities in cyberphysical systems security: A physics-based perspective, *IEEE Secur. Priv.* 18 (6) (2020) 14–22, <http://dx.doi.org/10.1109/MSEC.2020.3002851>.
- [4] S.S. Hussain, S.M. Farooq, T.S. Ustun, A method for achieving confidentiality and integrity in IEC 61850 GOOSE messages, *IEEE Trans. Power Deliv.* 35 (5) (2020) 2565–2567, <http://dx.doi.org/10.1109/TPWRD.2020.2990760>.
- [5] M.E. Hariri, T. Youssef, E. Harmon, H. Habib, O. Mohammed, The iec 61850 sampled measured values protocol: Analysis, threat identification, and feasibility of using nn forecasters to detect spoofed packets, *Energies* 12 (19) (2019) 3731, <http://dx.doi.org/10.1109/EEEEIC.2019.8783253>.
- [6] J. Olijnyk, B. Bond, J. Rrushi, Design and emulation of physics-centric cyberattacks on an electrical power transformer, *IEEE Access* 10 (2022) 15227–15246, <http://dx.doi.org/10.1109/ACCESS.2022.3148046>.
- [7] Schweitzer Engineering Laboratories, A Three-Phase Transformer Protection, Automation, and Control System, SEL-487E-3, -4 Data Sheet, Schweitzer Engineering, 2021, URL: https://cdn.selinc.com/assets/Literature/Product%20Literature/Data%20Sheets/487E-3-4_DS_20210708.pdf?v=20210723-164841.
- [8] S. Mazepa, L. Dostálek, O. Sharmar, S. Banakh, Cybercrime and vulnerability of Ukrainian critical information infrastructure, in: *2020 10th International Conference on Advanced Computer Information Technologies, ACIT, IEEE*, 2020, pp. 783–786.
- [9] Global Research & Analysis Team, Blackenergy apt Attacks in Ukraine Employ Spearphishing With Word Documents, Report, Securelist by Kaspersky, 2021, URL: <https://securelist.com/blackenergy-aptattacks-in-ukraine-employ-spearphishing-with-word-documents/73440/>.
- [10] M. Zeller, Myth or reality — Does the aurora vulnerability pose a risk to my generator? in: *2011 64th Annual Conference for Protective Relay Engineers*, 2011, pp. 130–136, <http://dx.doi.org/10.1109/CPRE.2011.6035612>.
- [11] M.E. El-Hawary, Introduction To Electrical Power Systems, in: *IEEE Press Series on Power Engineering*, vol. 50, Wiley, 2008, URL: <https://books.google.com/books?id=eLWh9w8ZpIC>.
- [12] P. Paganini, ICS/SCADA threats and threat actors, in: *Critical Infrastructure Blog and Topics*, Infosec, 2021, URL: <https://resources.infosecinstitute.com/topic/ics-scada-threats-and-threat-actors/>.
- [13] Clarity research team, Clarity Biannual ICS Risk & Vulnerability Report: 2H 2020, Clarity Biannual ICS Risk & Vulnerability Report, Clarity, 2020, URL: <https://security.clarity.com/biannual-ics-risk-vulnerability-report-2H-2020>.
- [14] S.S. Mack Grady, Understanding Power System Harmonics, University of Texas, Austin, TX, 2006, <http://dx.doi.org/10.1109/MPER.2001.961997>.
- [15] J. Guinn, L. Luque, J. Ray, Managing malware crashoverride/industrial malware assessment, Fireeye Threat Res. Blog (2017) URL: https://www.accenture.com/_acnmedia/PDF-69/Accenture-Managin-Malware-Crash-Override.pdf.
- [16] S. Miller, N. Brubaker, D.K. Zafra, D. Caban, Triton actor TTP profile, custom attack tools, detections, and ATT&CK mapping, Fireeye Threat Res. Blog (2019).
- [17] C. Parampalli, R. Sekar, R. Johnson, A practical mimicry attack against powerful system-call monitors, in: *Proceedings of the 2008 ACM Symposium on Information, Computer and Communications Security, ASIACCS '08*, Association for Computing Machinery, New York, NY, USA, 2008, pp. 156–167, <http://dx.doi.org/10.1145/1368310.1368334>.
- [18] R. Gopika, S. Deepa, Study on power transformer inrush current, in: *National Conference on Emerging Research Trends in Electrical, Electronics & Instrumentation (ERTEEI'17)*, IOSR Journal of Electrical and Electronics Engineering, Vol. 2, IOSR-JEEE, 2017, pp. 2278–1676.
- [19] F. Pedregosa, Plot memory usage as a function of time, 2014, URL: <https://fa.bianp.net/blog/2014/plot-memory-usage-as-a-function-of-time/>.
- [20] S. Adepu, N.K. Kandasamy, J. Zhou, A. Mathur, Attacks on smart grid: power supply interruption and malicious power generation, *Int. J. Inf. Secur.* (2019) 1–23, <http://dx.doi.org/10.1007/s10207-019-00452-z>.
- [21] Y. Kwon, H.K. Kim, Y.H. Lim, J.I. Lim, A behavior-based intrusion detection technique for smart grid infrastructure, in: *2015 IEEE Eindhoven PowerTech*, 2015, pp. 1–6, <http://dx.doi.org/10.1109/PTC.2015.7232339>.
- [22] Y. Kwon, S. Lee, R. King, J.I. Lim, H.K. Kim, Behavior analysis and anomaly detection for a digital substation on cyber-physical system, *Electronics* 8 (3) (2019) <http://dx.doi.org/10.3390/electronics8030326>, URL: <https://www.mdpi.com/2079-9292/8/3/326>.
- [23] M.E. Hariri, T. Youssef, E. Harmon, H. Habib, O. Mohammed, The IEC 61850 sampled measured values protocol: Analysis, threat identification, and feasibility of using NN forecasters to detect spoofed packets, in: *2019 IEEE International Conference on Environment and Electrical Engineering and 2019 IEEE Industrial and Commercial Power Systems Europe, IEEEIC / I&CPS Europe*, 2019, pp. 1–6, <http://dx.doi.org/10.1109/IEEEIC.2019.8783253>.
- [24] O. Mohamed, C.B. Chirila, Towards malware detection based on performance counters using deep learning classification models, in: *2022 IEEE 16th International Symposium on Applied Computational Intelligence and Informatics, SACI*, 2022, pp. 000149–000154, <http://dx.doi.org/10.1109/SACI55618.2022.9919602>.
- [25] C. Vaas, J. Happa, Detecting disguised processes using application-behavior profiling, in: *2017 IEEE International Symposium on Technologies for Homeland Security, HST*, 2017, pp. 1–6, <http://dx.doi.org/10.1109/THS.2017.7943508>.
- [26] G. Rodola, Cross-platform lib for process and system monitoring in Python, 2022, URL: <https://pypi.org/project/psutil/>.
- [27] L. Garcia, F. Brasser, M. Cintuglu, A.R. Sadeghi, O. Mohammed, S. Sonouz, Hey, my malware knows physics! attacking PLCs with physical model aware rootkit, 2017, <http://dx.doi.org/10.14722/ndss.2017.23313>.
- [28] A. Khanin, Time series and how to detect anomalies in them: part III, *Akvelon Blog* (2020) URL: <https://akvelon.com/time-series-and-how-to-detect-anomalies-in-them-part-iii/>.



Hussam Tarazi is currently doing Ph.D. in computer science with concentration on Cybersecurity and cyber physical systems security at Oakland University in Rochester, Michigan, USA, holds a masters degree in Information, network and computer security from New York institute of technology (2011) and Jordan university of science and technology, and also a bachelor in Software engineering with honors from The Hashemite university in Jordan (2007).

Worked in both the Information technology and academic domains for about 10+ years, served as a senior network and technical support administrator at Amman chamber of Industry and worked as a professional IT instructor and trainer at King Saud University and the National Information technology center of Jordan, and holds many professional IT and leadership certifications.

Recently, have been working with Dr.Rrushi in his research group for about 3 years doing research about malware detection and countermeasures for Industrial systems, also worked as a graduate research and teaching assistant at the department of computer science at Oakland university since 2019.



Sara Sutton received the Ph.D. degree in Computer Science and Informatics from Oakland University, USA, in 2020. She is currently Assistant professor at the school of computing at Grand Valley State University. Her research interests include cybersecurity, operating systems, and machine learning.



John Olijnyk is currently a Ph.D. student in computer science at Oakland University in Rochester, Michigan, USA. He has completed a Masters of business administration (2004) and a Masters of Science in electronics and computer control systems engineering (1998) from Wayne State University in Detroit, Michigan, USA. He also has a Bachelor of Science in electrical engineering (1994), computer science (1994), and mathematics (1994) with a minor in physics also from Wayne State University.

He has worked in industry for 25+ years including IBM, Detroit Diesel Corporation, Ford Motor Company, Idaho National Laboratory, and Cognizant Mobility in Troy, Michigan in various technical and leadership capacities in

the automotive, product development, manufacturing, and information technology domains. His research interests are focused on cyber physical system and software security engineering. Mr. John Olijnyk was awarded a US Patent for a Vehicle Convertible Track System: US6530621, Howard Donnelly Award for the top graduate of Wayne State University in leadership, service, and academics, Wayne State Board of Governors' Certificate of Appreciation, Stephen P. Hepler Computer Science Academic Award, Detroit Chapter of the National Society of Professional Engineers Student of the Year Award.



Benjamin Bond Benjamin was born in Waverly, Iowa, USA in 1990. He received the B.S. in philosophy from Iowa State University in 2014, the M.S. degree in information from the University of Michigan, Ann Arbor, and the M.S. degree in computer science from Oakland University, Rochester, Michigan, in 2017 and 2021 respectively. From 2019 to 2021, he was a Research Assistant with the Cybersecurity Lab at Oakland University, under the supervision of Dr. Julian Rrushi. Since 2021, he has been a Ph.D. student in computer science at Purdue University, West Lafayette, Indiana, USA, and a Microkernel Development Intern in the Critical Infrastructure Security group at the Idaho National Lab, Idaho Falls, Idaho, USA. His research interests are in the area of computer and network security, in particular secure microkernels, embedded systems security, and cryptography.



Julian Rrushi received the B.S. degree in computer science, the M.S. degree in information technology, and the Ph.D degree in computer science from the University of Milan. His Ph.D. work focused on detection of malware and exploits in cyber physical systems. He is currently with the Faculty of Engineering, Oakland University, MI, USA. Prior to commencing his academic career, he was a Senior Vulnerability Researcher with industry for several years. His research projects are supported by the U.S. Department of Defense, the U.S. Department of Energy, and industry. His research interests include intersection of operating systems, computer architectures, data science, and artificial intelligence to break new ground in cybersecurity. He was a recipient of the Young Faculty Award from DARPA, class of 2020, for research on the cyber physics of computing machines and networks.